

Universidad Nacional San Antonio Abad del Cusco
Escuela Profesional de Ingenieria Informatica y de Sistemas

Laboratorio Nro. 06

Analizador Sintactico LR con Flex y Bison

Curso: IF454AIN - Compiladores
Profesor: Victor Dario Sosa Jauregui
Estudiante: Efrain Vitorino Maric
Codigo: 160337
Repositorio: <https://github.com/lovito99/LaboCompiladores.git>

Índice

1. Introduccion	2
2. Objetivos	2
3. Archivos del proyecto	2
4. Fragmentos de codigo	2
4.1. Reconocimiento de numeros decimales positivos	2
4.2. Precedencia y asociatividad	3
4.3. Modulo o resto	3
4.4. Potencia	3
4.5. Division por cero	4
5. Compilacion	4
6. Pruebas de ejecucion	4
6.1. Pruebas correctas	4
6.2. Verificacion de jerarquia	5
7. Informe de errores	5
8. Conclusiones	5
9. Repositorio	5

1. Introduccion

El presente informe describe la implementacion del Laboratorio 6, que consiste en un analizador sintactico LR construido con Flex y Bison. El programa reconoce expresiones aritmeticas, aplica jerarquia de operadores, evalua cada expresion y reporta errores lexicos, sintacticos y matematicos.

Las mejoras incluidas son el operador modulo o resto, el operador potencia, la aceptacion de numeros decimales positivos usando `atof`, y el manejo de errores para division por cero y cero elevado a cero.

2. Objetivos

- Reconocer expresiones aritmeticas mediante un analizador lexico y sintactico.
- Implementar los operadores suma, resta, multiplicacion, division, modulo y potencia.
- Verificar la jerarquia y asociatividad de operadores.
- Aceptar numeros enteros y decimales positivos.
- Reportar errores matematicos para division por cero, modulo por cero y cero potencia cero.

3. Archivos del proyecto

Archivo	Descripcion
<code>lexer.l</code>	Analizador lexico. Reconoce numeros, operadores, parentesis y saltos de linea.
<code>analizador.y</code>	Analizador sintactico. Define gramatica, precedencia, acciones semanticas y errores.
<code>Makefile</code>	Automatiza la compilacion, ejecucion y limpieza del proyecto.
<code>informe/</code>	Carpeta que contiene el documento LaTeX y el PDF del informe.

4. Fragmentos de codigo

4.1. Reconocimiento de numeros decimales positivos

El analizador lexico reconoce enteros positivos, decimales con parte entera y decimales que inician con punto. El texto reconocido se convierte a `double` usando `atof`.

```
1 [0-9]+(\\. [0-9]+)?|\\. [0-9]+ {
2     yylval.valor = atof(yytext);
3     return NUM;
4 }
```

4.2. Precedencia y asociatividad

La jerarquia se define directamente en Bison. Las declaraciones escritas despues tienen mayor prioridad.

```
1 %left '+' '-'          /* menor prioridad */
2 %left '*' '/' '%'      /* prioridad media */
3 %right UMINUS          /* signo menos unario */
4 %right POTENCIA        /* mayor prioridad */
```

Con esta configuracion, la potencia tiene mayor prioridad que el producto, division, modulo, suma y resta. Ademas, POTENCIA es asociativa por la derecha, por lo que $2**3**2$ se interpreta como $2**(3**2)$.

4.3. Modulo o resto

El operador modulo se evalua usando fmod, funcion de la libreria matematica.

```
1 | expr '%' expr
2 {
3     if ($3 == 0) {
4         errores_sintacticos++;
5         error_matematico = 1;
6         fprintf(stderr, "Linea %d: Error matematico. Modulo por
7             cero.\n",
8             @3.first_line);
9         $$ = 0;
10    } else {
11        $$ = fmod($1, $3);
12    }
13 }
```

4.4. Potencia

La potencia se evalua con pow. Se agrega una validacion especial para el caso $0**0$.

```
1 | expr POTENCIA expr
2 {
3     if ($1 == 0 && $3 == 0) {
4         errores_sintacticos++;
5         error_matematico = 1;
6         fprintf(stderr,
7             "Linea %d: Error matematico. Cero elevado a cero
8             no esta definido.\n",
9             @2.first_line);
10        $$ = 0;
11    } else {
12        $$ = pow($1, $3);
13    }
14 }
```

4.5. Division por cero

La division valida el divisor antes de calcular el resultado.

```
1 | expr '/' expr
2 {
3     if ($3 == 0) {
4         errores_sintacticos++;
5         error_matematico = 1;
6         fprintf(stderr, "Linea %d: Error matematico. Division por
7             \ncero.\n",
8                 @3.first_line);
9         $$ = 0;
10    } else {
11        $$ = $1 / $3;
12    }
```

5. Compilacion

La compilacion se realiza desde la carpeta laboratorio-6.

```
1 make clean
2 make all
```

Salida obtenida:

```
1 rm -f analizador analizador.tab.c analizador.tab.h lexer.yy.c
   analizador.output
2 bison -d analizador.y -o analizador.tab.c
3 flex -o lexer.yy.c lexer.l
4 gcc -Wall -Wextra -g analizador.tab.c lexer.yy.c -o analizador -lm
5 rm -f analizador.tab.c analizador.tab.h lexer.yy.c
```

6. Pruebas de ejecucion

6.1. Pruebas correctas

Entrada:

```
1 10%3
2 10.5%4
3 2**3**2
4 -2**2
5 (-2)**2
6 1.5+2.25
```

Salida:

```
1 Resultado = 1
2 Resultado = 2.5
3 Resultado = 512
```

```

4 Resultado = -4
5 Resultado = 4
6 Resultado = 3.75

```

6.2. Verificacion de jerarquia

Expresion	Interpretacion	Resultado
2**3**2	2**(3**2)	512
-2**2	-(2**2)	-4
(-2)**2	Parentesis cambian la prioridad	4
1.5+2.25	Suma de decimales	3.75

7. Informe de errores

Entrada:

```

1 1/0
2 0**0
3 10%0

```

Salida:

```

1 Linea 1: Error matematico. Division por cero.
2 Linea 2: Error matematico. Cero elevado a cero no esta definido.
3 Linea 3: Error matematico. Modulo por cero.

```

Estos errores no imprimen resultado numerico para la expresion incorrecta. El programa marca el error matematico y continua con la siguiente linea de entrada.

8. Conclusiones

Se implemento el analizador sintactico LR con soporte para numeros decimales positivos, modulo y potencia. La jerarquia de operadores se controla mediante declaraciones de precedencia de Bison. Tambien se agrego manejo de errores matematicos para evitar resultados invalidos en division por cero, modulo por cero y cero elevado a cero.

9. Repositorio

El codigo fuente del laboratorio se encuentra disponible en:

<https://github.com/lovito99/LaboCompiladores.git>